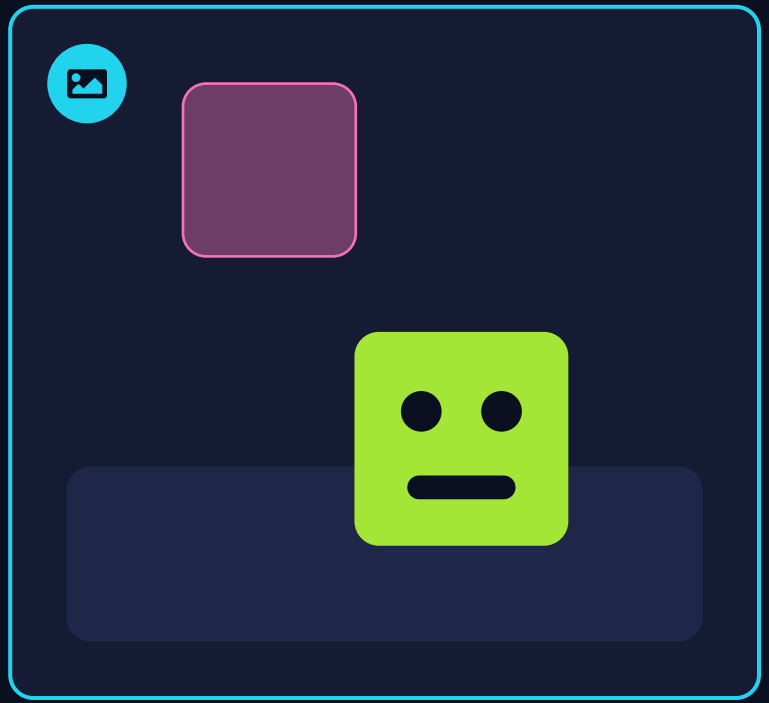
CCA GAME DEVELOPMENT

Loading Assets (Images)

Module 3 — Session 9

Teacher: Annisa Hendrawan

Junior High School — Global Prestasi School

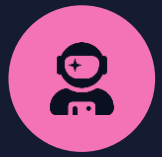


Today's Goal

By the end of this session, you will be able to:



Load image files (.png / .jpg) into Pygame.



Replace basic shapes with real character graphics.



Add a background image to your game.



Display and position images on the game screen.

Today's Mission: Turn a simple game into a game with real graphics!

From Shapes to Sprites

We used basic shapes to represent game objects.

```
pygame.draw.rect(
    screen,
    (70, 180, 255),
    (player_x, player_y, 50, 50)
)
```



But what if we want an actual character instead of a blue rectangle?

Today: We will replace rectangles with images!

What Are Game Assets?

Game Assets are the visual and audio files used in a game.

TODAY'S FOCUS: IMAGE ASSETS



Character / Sprite

The image representing our player.



Backdrop / Background

The image behind our game objects.



Enemy

An image representing an opponent or obstacle.

COMMON IMAGE FORMATS

`.png`	Supports transparent backgrounds.
`.jpg` / `.jpeg`	Commonly used for backgrounds and photographs.

Organizing Your Game Files

Before loading images, let's organize our project!

```
DodgeTheBlock/
│
├── main.py
│
└── assets/
    ├── player.png
    └── background.png
```

IMPORTANT!

- Keep your images inside the assets folder.
- Make sure the filenames are correct.
- Python needs the correct file path to find your images.

Loading an Image

Use pygame.image.load() to load an image.

```
player_image = pygame.image.load(
    "assets/player.png"
)
```

`pygame.image.load()` reads an image file.

`"assets/player.png"` tells Python where the image is.

`player_image` stores the loaded image.



Loaded… but not on screen yet!

Remember: Loading an image does not automatically display it!

Displaying Your Sprite

To display an image, use screen.blit().

```
screen.blit(
    player_image,
    (player_x, player_y)
)
```

`screen` = the game window.

`player_image` = the image we want to display.

`(player_x, player_y)` = the image position.



`blit()` draws the image onto the screen.

Adding a Background

We can load a background image in the same way.

```
background = pygame.image.load(
    "assets/background.png"
)
```

Then display it inside the game loop:

```
screen.blit(background, (0, 0))
```

Tip: Draw the background first!

DRAW ORDER

1 BACKGROUND



2 PLAYER



3 OTHER OBJECTS



Let's Replace Our Shapes!

PREVIOUSLY

```
pygame.draw.rect(
    screen, (70, 180, 255),
    (player_x, player_y, 50, 50)
)
```

NOW

```
screen.blit(
    player_image,
    (player_x, player_y)
)
```

And instead of filling the screen with one color:

```
screen.fill((30, 30, 45))
```

Use:

```
screen.blit(background, (0, 0))
```

Our goal: Keep the same game logic, but change how the game looks!

Let's Code!

Mini Mission: Upgrade Your Dodge the Block Game!

1. Find a character image (player.png).
2. Find a background image (background.png).
3. Create an assets folder.
4. Load both images using pygame.image.load().
5. Display the background using screen.blit().
6. Replace the player rectangle with the character image.
7. **Make sure the player can still move left and right!**

VS CODE · PYTHON

Run → Move LEFT / RIGHT → Observe

Challenge Time!

Optional Challenge: Can you customize your game?

- Change your character image.
- Try a different background.
- Find an image with a transparent background.
- Add an enemy image instead of a red rectangle.

BONUS QUESTION

What happens if your image is too big for the game window?

Test → Observe → Explain

What Did We Learn Today?

Game assets are files used in a game.

pygame.image.load() loads an image.

screen.blit() displays an image.

Image paths must be correct.

Backgrounds should be drawn before characters.

NEXT SESSION 10: THE MAGIC OF HITBOXES (RECTS)

We will learn how to create invisible rectangles around our game objects!